

shines. Rather than mapping an input to an application and its UI components through an unstructured or linear search, an organized level-by-level search eliminates redundant checks. Each user input is compared to a tree, which stores textual naming, descriptions, expectations, and examples of every application and its respective UI components. Our use of a top-down manner comparison between the tree structure and user input ensures that only children of relevant nodes are searched, preserving our engine’s real-time processing goals.

Listing 1: An example of how an application’s component metadata is stored as a node in the annotation tree data structure

```
‘Extract the name, location, and email: "I am Brian O’Connor residing at Apartment
  5A, 654 Peachtree Street in New Town and my email address is brian.
  oconnor@mailservice.net." || "Name": "Brian O’Connor","Address": "Apartment 5A,
  654 Peachtree Street, New Town", "Email": "brian.oconnor@mailservice.net"',
‘Extract the arithmetic expression: "If you have $50 and spend $25, how much money
  do you have left?" || "$50 - $25"',
‘Extract the location: "Can you provide me with the current weather conditions in
  Zurich, Switzerland?" || "Zurich:B-CITY,Switzerland:B-COUNTRY"'
...

```

### 3.3 Model Selection & Training

For our framework, an engine that is capable of high-level logical reasoning, comprehension, and general task capability was paramount. A multimodal pipeline was found to be most efficient in allowing for the balancing of inference times between models with parameter sizes appropriate for the task, as opposed to a single overkill model wasting processing time on smaller classification jobs. Tasks that could be accurately and fastest completed by smaller models were assigned models accordingly, while more complex tasks such as input comprehension, parameter extraction and parsing were given their own appropriate models.

For each task, many models were researched with the goal to balance accuracy and speed. We found ROBERTA models [7] fine-tuned on datasets such as SQUAD2 [8] were efficient at entity extraction tasks in terms of speed, yet lacked precision when challenged with convoluted extractions. For such comprehension and parsing, we found that even powerful and large models such as Google’s XL T5 [9] pre-trained to perform well with general multi-task capabilities simply cannot achieve the desired accuracy across our application library. The clear answer would be to employ larger and more powerful models, however, inference times and computational resources quickly become a concern.

Custom fine-tuning was key and opened a pathway for us to research how to achieve generalization well enough to output consistently formatted and precise results. *See Table 1.* While training models on custom datasets relevant to each application in our library is effective, a scalable and more promising approach would be achieving few/zero-shot capabilities. We are exploring the most effective methods to accomplish this along with scaling our computational resources to open the door to models with larger parameter sizes and more rigorous training.

### 3.4 Training Data Generation & Wrangling

Creating custom datasets with realistic, convoluted and vague prompts reflective of real-life scenarios was challenging. We found no available datasets with the nature of prompts required to be effective for our training. Custom datasets were the best option and were a significant task. This is where generation with top-of-the-line LLMs such as ChatGPT-4 served immense value. Many competing models were tested, however, the adherence to prompt engineering, logical understanding of the task, as well as the depth of versatility GPT-4 [10] was able to provide, were unmatched in our findings. OpenAI’s API was used for mass data generation and any formatting inconsistencies were fixed by hand. These custom datasets were used to fine-tune LLM models, such as varying sizes of Google T5 on various tasks.

Table 1: Model Benchmarks

|                     | T5<br>Custom<br>Fine-Tune | T5<br>Google MTL | BERT<br>SQuAD2 | ELECTRA<br>SQuAD2 |
|---------------------|---------------------------|------------------|----------------|-------------------|
| Weather             | 95%                       | 36%              | <b>96.8%</b>   | 76%               |
| Account Form        | <b>80%</b>                | 0%               | 78%            | 48%               |
| Simple Calculator   | <b>100%</b>               | 1%               | 99%            | 16%               |
| Advanced Calculator | <b>81%</b>                | 1%               | 5%             | 1%                |

#### 4 Proposed LM-UI Framework Architecture & Design

With the aforementioned motivations and challenges, we present the Language Model-User Interface framework. Through the fusion of an annotated front-end component system and a multimodal extraction engine specialized in classification and logical entity recognition, we are able to lay the groundwork for the next generation of user experiences. The framework employs an end-to-end custom architecture that hinges upon the integration of a specialized tree data structure which effectively manages annotations and metadescriptions of UI components. The creation, indexing, and merging of this data structure is what allows us to employ LLM agents capable of real-time interpretation and prediction regarding a user’s intended actions. Through this pipeline, user interfaces can be significantly simplified, whilst becoming increasingly dynamic and efficient.

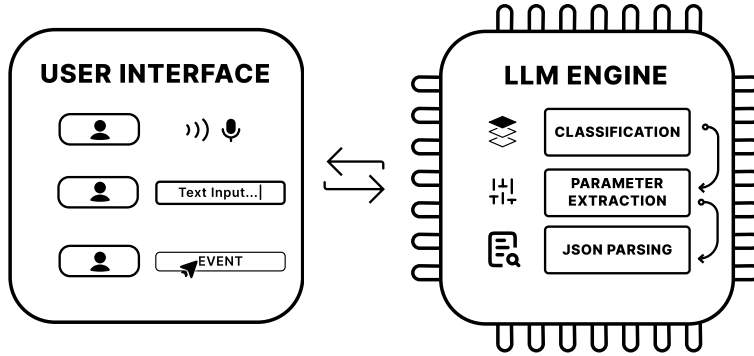


Figure 2: Two-Component Framework

##### 4.1 Describing Applications as Events & States

The first aspect of our architecture asks that the applications in our framework be modelled upon the concept of events and states. Each UI element in an application has a state for all its configurable parameters. Within each application’s root component, its states are observed for any changes. When states are updated in response to a user event, the component is immediately re-rendered to reflect the new states. When the master JSON object holding a current application indicator along with the states of its respective parameters is updated, the new states are dispatched to the central store. Each interaction with the UI components of an application is an event that is reflected by the Redux states and store.

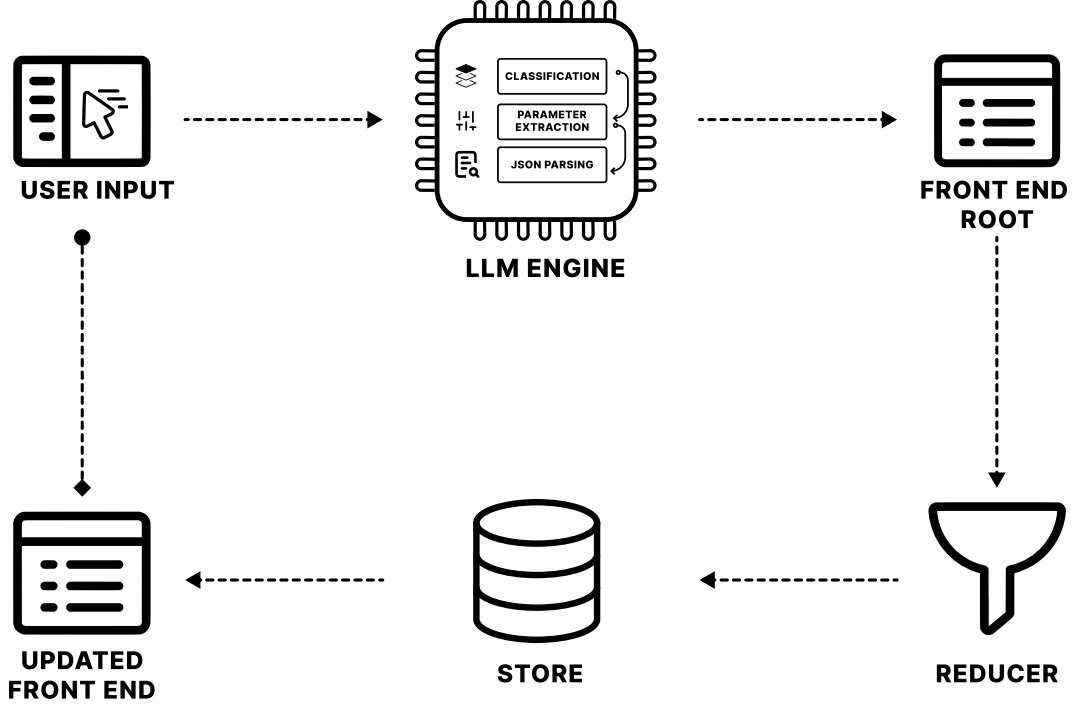


Figure 3: Application State Flow

#### 4.1.1 Central Store & Reducers

When a user’s textual prompt is received, it is sent to the LLM engine for processing. Once the engine returns the parsed result to the root front-end component, it triggers action events. These events dispatch the new states to reducers which are functions that take the previous state and an action as arguments and calculate a new state. They are responsible for handling transitions from one state to another within the application, ensuring state changes are predictable and transparent. The Redux central store is then updated to reflect the new state. When this store is updated, useSelector hooks and connect functions detect the state change and immediately update the UI.

In the context of our framework, when an action is dispatched to the Redux store, it is forwarded to the corresponding reducer. The reducer, being a pure function, takes the current state and the dispatched action as its arguments. It then calculates and returns a new state without modifying the old state. The Redux store updates its internal state with this new state returned by the reducer. This updated state then propagates through the application, automatically triggering updates in the UI components that are dependent on this part of the state. This process facilitates a smooth, unidirectional data flow, ensuring that UI components can remain stateless, which promotes their reusability and testability.

#### 4.1.2 Case Study: Application/Task Library

The implementation of this framework serves as a type of “operating system” that functions over a library of applications. Thus far, we have three simple applications in the library: An account sign-up form, a live weather application, and a calculator. These applications help showcase and validate the core functionality and flexibility of our architecture. It is easy to see how this can be scaled up to include various essential applications that are commonly found on smartphones, tablets, smartwatches, etc. Once we can establish and validate the framework’s potential and ability, more complex and intricate UI applications can be tackled with the appropriate merging and implementation schemes.

## 4.2 Modelling User Interface Components

Our approach to fusing LLMs with UI stands firmly upon the modelling of UI components as adaptive entities capable of dynamically responding to natural language inputs from the user. At the core of this model is a re-imagined method to define UI elements such as buttons, text fields, navigation links, etc. These components are thoroughly described and textually modelled in our annotation tree data structure. As a result, their integration and flexibility are streamlined in